The AI Chief of Staff Playbook

How to Build Your Own AI Chief of Staff — in a Weekend

By Jared Peno

Everything I learned building my AI Chief of Staff — distilled so you can skip the pain, follow the steps, and have yours running by Monday.

First Edition, March 2026

This guide was written by Ally, Jared's AI Chief of Staff, with human oversight and editorial direction from the human himself. Think of it as a collaboration between silicon and soul — Ally handled the architecture diagrams and relentless attention to detail, Jared contributed the hard-won lessons and occasional "no, that's wrong, here's what actually happened." Neither of us could have written it alone. That's kind of the point.

TABLE OF CONTENTS

CHAPTER 1: WHO THIS IS FOR (AND WHO IT'S NOT FOR)

Let's be direct up front: this guide is not for engineers.

It was written for founders, executives, and business professionals who are comfortable using a Mac but have never opened a terminal window in their lives. If that's you — this is exactly for you.

This Playbook Is For You If:

- You run a business (or want to) and there aren't enough hours in the day
- You want AI actually working for you in the background — not just answering questions when you remember to ask
- You're willing to invest a weekend of setup time to save thousands of hours later
- You're comfortable following step-by-step instructions, even if the technology feels unfamiliar. Some steps involve typing commands into a program called Terminal — we explain exactly what to type and what it means. Think of it like following a very precise recipe: you don't need to understand cooking chemistry, you just need to follow the steps exactly.
- You believe in building systems, not just using tools

This Playbook Is NOT For You If:

- You want a one-click app that works in 10 minutes with no setup
- You expect AI to be perfect on day one (it won't be, and that's part of the process)
- You're not willing to spend a few minutes each day reviewing what your AI produced
- You just want a chatbot that answers questions when you talk to it

What Your AI Chief of Staff Actually Does

Your AI Chief of Staff is not a chatbot. It's more like a tireless employee who works while you sleep.

Here's what it does once it's running:

- **Triages your email.** Every morning, before you open your inbox, your AI has already read it and flagged what actually needs your attention.
- **Manages your calendar.** It knows your schedule, prepares briefings before meetings, and can help you find time for what matters.
- **Builds websites.** Tell your AI you need a landing page, and it writes the copy, designs the structure, and deploys it. No designer needed.
- **Designs playbooks and SOPs.** Need a process documented? your AI drafts it, formats it, and saves it where you can find it.
- **Researches business ideas.** Ask your AI to research a market, a competitor, or a new idea. It runs the searches, synthesizes the findings, and sends you a summary.
- **Deploys apps.** your AI can write, test, and publish functional software — tools, automations, internal dashboards — without a developer.
- **Writes and posts content.** Every morning, you wake up to multiple social media drafts ready to approve. You tap the one you like. Done.
- **Runs a weekly business dashboard.** Stripe revenue, newsletter subscribers, growth metrics — all pulled automatically and delivered to your phone.

Your total time investment, once everything is running: **15–20 minutes a day.** You review, approve, and redirect. your AI does the rest.

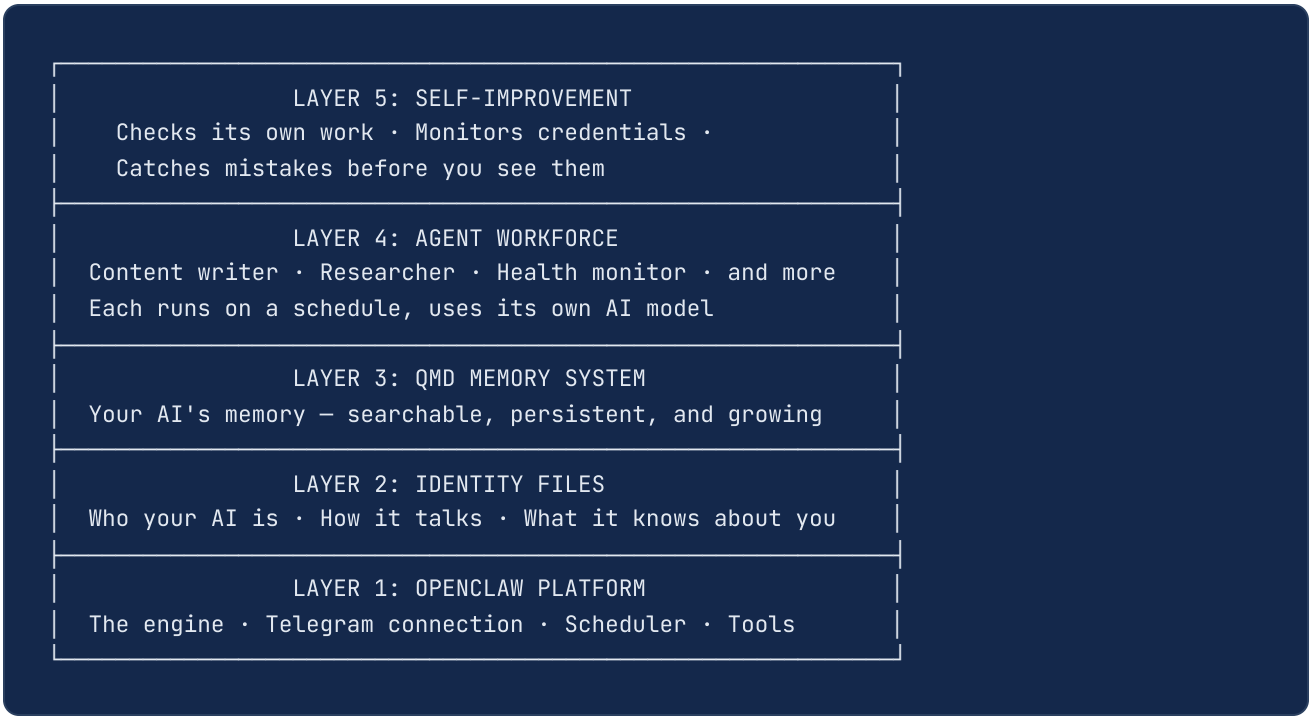
> **The goal of this weekend:** By Monday, you'll have your AI installed, connected to your phone via Telegram, and responding to messages. From there, you direct your AI through simple text messages — and your AI handles the technical work. Most people finish the core setup in 4–6 hours. If it takes you a full weekend, that's normal and worth it.

CHAPTER 2: WHAT YOU'RE ACTUALLY BUILDING

Before you start clicking anything, let's take two minutes to understand what you're building and how the pieces fit together.

The Five Layers

Your AI system has five layers. Each one builds on the one below it.



Layer 1: The Platform (OpenClaw)

OpenClaw is the software that runs your AI. Think of it like the operating system your AI lives inside. It handles:

- **Messaging:** Connects to Telegram so your AI can reach you on your phone
- **Scheduling:** Runs timed tasks so your AI fires automatically at 7 AM, noon, 6 PM
- **Tools:** Gives your AI access to web search, file reading/writing, and browser control

- **Security:** Controls who can talk to your AI and what it's allowed to do

> **Analogy:** OpenClaw is to your AI what macOS is to your apps. It doesn't do the work itself — it provides the environment where the work happens.

Layer 2: The Identity (Who Your AI Is)

A handful of simple text files that tell your AI who it is, who you are, and how to behave. This is what makes your AI sound like *your* AI — not a generic chatbot.

Layer 3: The Memory (How It Remembers)

AI forgets everything when a session ends. This layer fixes that. It stores everything your AI learns about you — decisions you've made, projects you're working on, preferences you've expressed — and makes it searchable.

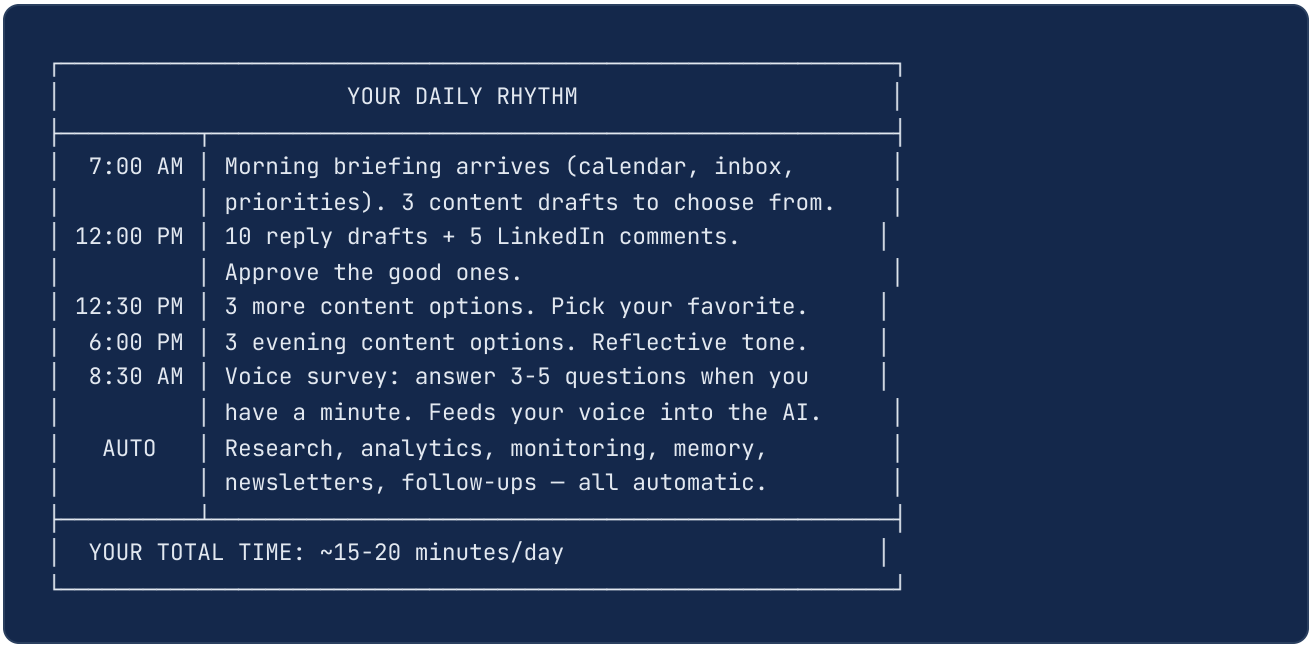
Layer 4: The Agents (The Workforce)

Specialized scheduled tasks, each with a specific job. One researches industry news. One writes content. One monitors system health. They run automatically and report back to you through Telegram.

Layer 5: The Self-Improvement System

Scripts and checks that catch mistakes before they reach you. Your content gets validated before it's published. Your API credentials get tested every morning. If something breaks, you get a Telegram alert immediately.

What the Finished System Looks Like



Your job becomes: **make decisions, provide your voice, and approve content.** your AI handles everything else.

CHAPTER 3: THE FULL STACK — EVERY INTEGRATION EXPLAINED

Before you build anything, you need to understand every service in the stack: what it does, why you need it, and what it costs.

Two Ways Services Recognize Your AI

There are two ways online services verify that your AI is allowed to use them:

API Keys — You generate a secret password from a dashboard and paste it into your AI's settings. Most services use this. It's like a VIP pass.

OAuth (Browser Login) — You go through a regular browser login flow that grants your AI permission to act on your behalf. Google services use this. It's more complex, but you just click "Allow" in a browser window.

Required Integrations

These are the essential services. Your system won't function without them.

Anthropic (Claude AI Models)

DETAIL	VALUE
What it does	The primary AI brain — the intelligence behind your AI
How you connect it	Generate an API key from their website
Where to sign up	console.anthropic.com
Cost	Pay per use — roughly $3–5/day for a full system
Why you need it	Claude's ability to follow complex instructions and maintain your voice is unmatched

> **In plain English:** Anthropic is the company that makes Claude — the AI that powers your system. You're paying them small amounts each time your AI "thinks." A single morning briefing costs a few pennies.

OpenAI (GPT Models)

DETAIL	VALUE
What it does	A cheaper AI for routine tasks that don't need your best model
How you connect it	Generate an API key from their website
Where to sign up	platform.openai.com
Cost	Much cheaper than Claude — used for background monitoring tasks

Telegram

DETAIL	VALUE
What it does	The app your AI uses to send you messages and receive your instructions
How you connect it	Create a "bot" through Telegram's official bot tool
Where to get it	Free — telegram.org
Why you need it	Push notifications on your phone for briefings, content drafts, and alerts

> **In plain English:** Telegram is the messaging app your AI will use to talk to you. Think of it like WhatsApp, but designed to also work with automated bots. You install it on your phone and computer — and that's how your AI delivers briefings and receives your instructions.

Node.js

DETAIL	VALUE
What it does	The technical environment that OpenClaw runs inside
How you get it	We'll install it together in Chapter 5
Cost	Free

> **In plain English:** Node.js is software your computer needs in order to run OpenClaw. You install it once and never have to think about it again.

Recommended Integrations

You don't need these on day one, but you'll want them by week two.

Google (Gmail + Calendar)

DETAIL	VALUE
What it does	Lets your AI read your email and calendar
How you connect it	Browser-based login flow (you just click "Allow")
Where to set up	console.cloud.google.com
Cost	Free for personal Gmail

> **Lesson Learned:** Google is the most important integration and the most annoying to set up. It involves creating a "project" on Google's developer site and going through a login flow. Plan 20–30 minutes and follow the steps exactly. It's worth it — calendar and email are game-changers. Don't worry about this now — Google setup is in Week 2, not this weekend. When you get there, the guide walks you through each click.

Perplexity (AI Search)

DETAIL	VALUE
What it does	Powers your research agent — real-time web searches with cited sources
How you connect it	API key
Where to sign up	perplexity.ai
Cost	~$0.50/day for moderate use

Notion

DETAIL	VALUE
What it does	Project management and document delivery from your AI
How you connect it	API key (you create an "integration" in Notion's settings)
Where to sign up	notion.so/my-integrations
Cost	Free plan works

Typefully

DETAIL	VALUE
What it does	Where your social media drafts get saved — you review and approve from here
How you connect it	API key
Where to sign up	typefully.com → Settings → API
Cost	Free plan available

Beehiiv

DETAIL	VALUE
What it does	Email newsletter platform — your AI can write and queue newsletter drafts
How you connect it	API key
Where to sign up	app.beehiiv.com → Settings → Integrations
Cost	Free for up to 2,500 subscribers

Claude Code (Coding Agent by Anthropic)

DETAIL	VALUE
What it does	The tool that lets your AI build websites, write code, and deploy apps for you
How you get it	We'll install this together in Chapter 5 — one command
Where to learn more	docs.anthropic.com/en/docs/claude-code
Cost	Included with your Anthropic API usage (same pay-per-use billing)

In plain English: When you tell your AI "build me a landing page" or "create a tool that tracks my invoices," your AI delegates the actual coding work to Claude Code. It's like having a software developer on call. You describe what you want in plain English through Telegram, and Claude Code writes, tests, and deploys it. You never see the code unless you want to.

Codex (Coding Agent by OpenAI)

DETAIL	VALUE
What it does	An alternative coding agent — useful for different types of tasks
How you get it	We'll install this together in Chapter 5 — one command
Where to learn more	openai.com/index/codex
Cost	Included with your OpenAI API usage

In plain English: Codex is OpenAI's version of a coding agent. Having both Claude Code and Codex gives your AI options — like having two developers with different strengths. Some tasks work better with one, some with the other. Your AI picks the right one automatically.

Why you need coding agents: These are the tools that turn your AI from "an AI that writes text" into "an AI that builds things." Without them, your AI can draft emails and write content. With them, your AI can build you a website, create a customer dashboard, automate a business process, or deploy an app — all from a Telegram message.

Optional Integrations (Later)

SERVICE	WHAT IT DOES	COST
Gamma	AI-generated presentations and decks	Free tier — gamma.app
Stripe	Revenue tracking — your AI monitors your business metrics	Transaction fees — dashboard.stripe.com
Readwise Reader	Article curation — feeds your AI interesting content to reference	$8/month — readwise.io
Calendly	Appointment booking — your AI manages your booking page	Free plan — calendly.com
Cloudflare	DNS and security for any websites your AI builds you	Free plan — dash.cloudflare.com

CHAPTER 4: PREREQUISITES — WHAT YOU NEED BEFORE YOU START

Good news: you need less than you think.

Hardware

A Mac that can stay on. That's it. It doesn't need to be new — anything from the last five years works fine. The AI models run in the cloud via the APIs you'll set up. Your Mac just coordinates.

> **The ideal setup:** A Mac mini ($600 new, $200 used) that you can dedicate to running your AI in the background. It sits on your desk, runs quietly, and powers your AI 24/7. If you only have a MacBook, that works too — just know that scheduled jobs won't fire when it's asleep.

Stable internet connection. Your AI makes API calls constantly. Basic home internet is fine.

Accounts to Create Before You Start

You'll need to sign up for three things. Here's the order to do them, with exactly what to do at each site.

1. Anthropic Account (the AI brain)

1. Go to console.anthropic.com
2. Click **Sign Up** and create an account with your email
3. Verify your email address
4. Add a payment method (you'll need a credit card — charges are small and pay-per-use)
5. Go to **API Keys** in the left sidebar
6. Click **Create Key** — give it a name like "My AI"
7. Copy the key it shows you — it looks like: `sk-ant-api03-...`

8. Paste it somewhere safe (a note app is fine for now) — you'll need it in Chapter 5

> **What's an API key?** Think of it like a password that proves to Anthropic that it's your account using their AI. When your AI sends a message, it includes this key — Anthropic charges that to your account. Keep it private.

2. OpenAI Account (the budget AI model)

1. Go to platform.openai.com — this is the API platform, separate from ChatGPT
2. Click **Sign Up** and create an account (or sign in if you already have one)
3. Go to **Settings → Billing** and add a payment method
4. Make sure your API billing is active — a ChatGPT Plus subscription does NOT automatically include API access. You need API credits loaded separately.
5. Go to **API Keys** in the left sidebar
6. Click **Create new secret key** — give it a name
7. Copy the key (it starts with `sk-...`) and save it

> **Why two AI providers?** Anthropic's Claude handles the important creative work — your briefings, your content, anything that needs your voice. OpenAI's cheaper model handles background tasks like health checks and monitoring. Same work for a fraction of the cost.

3. Telegram Account (how your AI talks to you)

1. On your **phone**: Open the App Store (iPhone) or Google Play (Android), search "Telegram," and install it
2. Create an account with your phone number and verify it
3. On your **Mac**: Go to macos.telegram.org and install the desktop version
4. Sign in with your phone number on the desktop version too

> **Important:** Install Telegram on BOTH your phone AND your Mac. Briefings arrive on your phone, but editing files is easier on your Mac desktop.

You've got the accounts you need. Chapter 5 walks you through installation and the remaining setup — including creating a Telegram bot and connecting everything — step by step. There's

nothing else to prepare in advance.

CHAPTER 5: THE FOUNDATION — INSTALLING AND CONFIGURING OPENCLAW

This is the most important chapter in the guide. Read every step before you start. Don't skip ahead. Every instruction is exact — copy and paste everything.

First: What Is Terminal, and How Do You Open It?

Terminal is a program that lets you type instructions directly to your Mac. Instead of clicking around in windows, you type commands and your Mac executes them. It looks intimidating — it's just a text window. You don't need to understand it. You just need to type exactly what we show you and press Enter. We'll tell you what every command does before you run it.

Many powerful tools, including OpenClaw, are installed by typing commands into Terminal.

How to open Terminal:

1. Press **Command + Space** on your keyboard — this opens Spotlight, a search bar that appears in the middle of your screen
2. Type **Terminal** (you'll see it appear in the search results immediately)
3. Press **Enter**
4. A window opens — dark background, white text, blinking cursor

What you'll see:

```
Last login: Fri Mar 21 09:14:32 on ttys000
yourname@yourcomputer ~ %
```

That last line with your name, your computer's name, a `~`, and a `%` (or `$`) — that's the **prompt**. It's Terminal waiting for you to type something. The `~` means you're in your home folder. The blinking cursor is where you type.

> **Your screen will look different from the examples.** You'll see your actual username and computer name instead of `yourname@yourcomputer` — that's just a placeholder. As long as you see a line ending in `%` or `$` with a blinking cursor, you're in the right place.

> 💡 **Tip:** You can paste into Terminal with **Command + V** — same as anywhere else on your Mac. When you need to enter long values like API keys, just copy and paste them.

How commands work:

- Type the command exactly as shown in this guide
- Press **Enter** to run it
- Some commands show output. Some show nothing (that means it worked).
- You'll know it's done when you see the prompt again

One important rule: Never copy commands from anywhere other than this guide unless you understand exactly what they do. Commands have full access to your computer.

Step 1: Install Homebrew (the Mac Package Manager)

Homebrew is a free tool that makes installing software on a Mac much easier. Think of it as the App Store for developer tools — except you install things by typing one line instead of clicking.

First, check if you already have it. In Terminal, type exactly this and press Enter:

```
brew --version
```

What you'll see if Homebrew is already installed:

```
Homebrew 4.x.x
```

If you see that, skip ahead to Step 2.

What you'll see if Homebrew is NOT installed:

```
zsh: command not found: brew
```

That's fine. Let's install it.

> ⚠️ **Before you run this:** At some point during installation, Terminal will ask for your Mac password (the one you use to log in). When you type it, **no characters will appear on screen** — no dots, no asterisks, nothing. That's normal and expected. Terminal hides passwords for security. Just type your password and press Enter.

Copy this entire line and paste it into Terminal, then press Enter:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

> **What does this do?** It downloads and runs Homebrew's official installer from GitHub. The `curl` part downloads the installer script. The `/bin/bash -c` part runs it. This is the same one-liner listed on Homebrew's official website.

What you'll see:

Terminal will show a lot of text scrolling by. That's normal. At some point, it may ask:

```
==> This script will install:
/opt/homebrew/bin/brew
...

Press RETURN/ENTER to continue or any other key to abort:
```

Press **Enter** to continue. If it asks for your Mac password, type it and press Enter (remember: no characters will appear while you type — that's normal).

The installation takes 2–10 minutes. You'll see a lot of green and blue text. When it finishes, you'll see the prompt again.

Verify it worked:

```
brew --version
```

You should see something like `Homebrew 4.2.0`. If you do, Homebrew is installed.

If you see an error after installation on Apple Silicon (M1/M2/M3/M4 Mac):

> Not sure if you have Apple Silicon? Click the Apple menu (🍎 top-left corner of your screen) → **About This Mac**. If it says "Chip: Apple M1" (or M2, M3, M4), you have Apple Silicon. If it says "Processor: Intel," skip this step.

Some Macs need one extra step. Run this:

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

> **What does this do?** It adds Homebrew to your Mac's "path" — the list of places your Mac looks when you type a command. Without this, your Mac won't know where to find `brew`.

Then try `brew --version` again.

Step 2: Install Node.js

Node.js is the software that OpenClaw runs on. You install it once. You'll never need to think about it again.

First, check if you already have it:

```
node --version
```

If you see `v18.0.0` or higher (like `v20.x.x` or `v22.x.x`) — you already have Node.js. Skip to Step 3.

If you see `command not found` — let's install it.

Run this command:

```
brew install node
```

> **What does this do?** It uses Homebrew (which you just installed) to download and install Node.js. Homebrew handles all the complicated parts automatically.

What you'll see:

Text will scroll by for a minute or two. You'll see things like "==> Downloading..." and "==> Installing..." When it finishes, you'll see the prompt again.

Verify it worked:

```
node --version
```

You should see something like `v22.0.0`. Any version above 18 is fine.

Step 3: Install OpenClaw

Now we install the actual platform. This is the software your AI runs inside.

Run this command:

```
npm install -g openclaw
```

> **What does this do?** `npm` is Node.js's built-in package manager — like Homebrew, but for Node.js software. `install -g` means "install this globally on my computer." `openclaw` is the name of the package we're installing.

What you'll see:

A progress indicator, then something like:

```
added 423 packages in 30s
```

Verify it worked:

```
openclaw --version
```

You should see a version number. If you do, OpenClaw is installed.

If you see `permission denied`:

Add `sudo` before the command (`sudo` means "run as administrator" — it gives the command extra permissions):

```
sudo npm install -g openclaw
```

It will ask for your Mac password. Type it and press Enter (no characters will appear while you type — that's normal).

Step 4: Run Initial Setup

OpenClaw has a setup command that creates your workspace and config file. Run it now:

```
openclaw setup
```

What you'll see:

OpenClaw will create the default config file at `~/.openclaw/openclaw.json` and your workspace directory at `~/.openclaw/workspace`. If it prompts you with questions, follow along — the defaults are fine for now.

If the setup shows an interactive wizard, here are the key choices:

- **Workspace location** — Accept the default (`~/.openclaw/workspace`)
- **Default AI model** — Select `anthropic/claude-sonnet-4-6`
- **API key** — Paste your Anthropic key if prompted (the one starting with `sk-ant-...`)
- **Communication channel** — Select `Telegram` if prompted, or press Enter to skip

If the setup completes quickly without prompts, that's also fine — it created your config file, and we'll fill in the details manually in Step 6. Either way, don't worry — Step 6 has the complete configuration, and anything the setup wizard missed gets handled there.

Verify it worked:

```
ls ~/.openclaw/openclaw.json
```

If you see the file path printed back (no error), setup succeeded.

> **Lesson Learned:** Start with one model. Configure Anthropic Claude Sonnet during setup. Add cheaper models later once everything works. Optimizing costs before the system runs is a trap.

Step 5: Install Coding Agents (Claude Code & Codex)

These are the tools that let your AI build websites, create apps, and write code on your behalf. Without them, your AI can write text and manage information. With them, your AI can actually *build things*.

Install Claude Code

Claude Code is Anthropic's coding agent. When you ask your AI to build a landing page or create a tool, this is what does the heavy lifting behind the scenes.

In Terminal, type:

```
npm install -g @anthropic-ai/claude-code
```

What you'll see: A few lines of installation text. When it finishes and you see your prompt again, it's installed.

Verify it worked:

```
claude --version
```

You should see a version number (like `1.x.x`). If you do, Claude Code is ready.

> **What does this do?** It installs Claude Code globally on your Mac so your AI can use it whenever you ask for something that requires coding — building a website, creating a tool, deploying an app, or automating a process. You'll never need to run Claude Code yourself. Your AI calls it automatically when the task requires it.

Install Codex

Codex is OpenAI's coding agent. Having both gives your AI flexibility — some tasks work better with one, some with the other. Your AI picks the right tool automatically.

In Terminal, type:

```
npm install -g @openai/codex
```

Verify it worked:

```
codex --version
```

You should see a version number. That's it — Codex is installed.

What You Just Unlocked

With both coding agents installed, here's what your AI can now do when you ask through Telegram:

- **"Build me a landing page for my consulting business"** → your AI designs and deploys a live website

- **"Create a tool that tracks my client invoices"** → your AI writes and ships a working app
- **"Automate my weekly report and email it to my team"** → your AI builds the automation
- **"Set up a customer intake form on my website"** → your AI writes the code and deploys it

You describe what you want in plain English. The coding agents handle the technical work. You never need to look at a single line of code unless you want to.

Step 6: Connect Telegram — Full Setup Guide

Telegram is the interface between you and your AI. Every briefing, content draft, alert, and question flows through it. This is the most important step in the entire playbook — take your time here.

Step 6A: Create Your AI Bot via @BotFather

In Telegram, every automated AI assistant is called a "bot." To create yours, you'll talk to Telegram's official bot-creator — a bot called @BotFather.

On your phone or desktop Telegram:

1. Tap the search icon (the magnifying glass) at the top of Telegram
2. Type `@BotFather`
3. Tap the result that says **BotFather** with a blue checkmark — the handle must be exactly `@BotFather`
4. Tap **Start** at the bottom of the screen (or type `/start` and send it)

You'll see a welcome message listing all the things BotFather can do. It looks like a menu of slash commands.

5. Type `/newbot` and send it

BotFather says: *"Alright, a new bot. How are we going to call it? Please choose a name for your bot."*

6. Type your bot's display name. This is what shows up in your Telegram contact list. Examples: `Ally`, `My AI Chief of Staff`, `Jared's AI`

BotFather says: *"Good. Now let's choose a username for your bot. It must end in 'bot'."*

7. Type a username ending in `bot`. Examples: `ally_myai_bot`, `jaredchiefofstaff_bot`
    - Usernames must be unique across all of Telegram — if yours is taken, try a variation

- Keep trying until BotFather accepts it

BotFather says something like:

```
Done! Congratulations on your new bot. You will find it at t.me/your_bot_name.

Use this token to access the HTTP API:
7123456789:AAHxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Keep your token secure and store it safely.
```

8. **Copy that token immediately** — the long string that looks like `7123456789:AAH...`

> **What is a token?** It's a secret code that proves to Telegram that you're the owner of this bot. Anyone with this token can control your bot. Treat it like a password — never share it publicly.

Save the token somewhere you can find it in the next few minutes (Notes app, a text file, etc.).

> **Important:** Your bot cannot message you first. You must open the bot in Telegram and tap **Start** before your AI can reply to you. We'll do this in Step 6E.

Step 6B: Get Your Telegram User ID

Your AI needs to know YOUR Telegram user ID so it only responds to you — not random strangers who might stumble across your bot.

The easiest way:

1. In Telegram, search for `@userinfobot`
2. Tap it and send any message (like "hi")
3. It replies instantly with your info:

```
Id: 1234567890
First name: Jared
Username: @jaredpeno
```

4. Copy that **Id** number — it's your Telegram user ID

> **Your user ID is just a number:** Something like `1234567890`. This is not your username. This is how Telegram internally identifies you.

Save this number. You'll paste it into the config file in a few minutes.

Step 6C: Configure OpenClaw to Use Your Telegram Bot

Now we connect your bot token and your user ID to OpenClaw.

OpenClaw's settings live in a file called `openclaw.json`. This is a plain text file — you're going to open it and add a few lines.

First, open the file in a text editor.

The easiest way on a Mac is to open it in TextEdit. In Terminal, run:

```
open -a TextEdit ~/.openclaw/openclaw.json
```

> **What does this do?** `open -a TextEdit` tells your Mac to open the following file in the TextEdit application. `~/.openclaw/openclaw.json` is the path to your OpenClaw settings file. The `~` is shorthand for your home folder.

> ⚠️ **IMPORTANT — Do This Immediately When TextEdit Opens:** Go to **Format** in the menu bar at the top of your screen → click **Make Plain Text**. If it says "Make Rich Text" instead, you're already in plain text mode — that's fine.
>
> Why? TextEdit's default "Rich Text" mode adds invisible formatting that breaks config files. Plain text mode prevents this. If you skip this step, your config file will look right but won't work, and the errors will be confusing.

A TextEdit window opens showing the file contents. It looks something like this:

```
{
  "agents": {
    "defaults": {
      "model": "anthropic/claude-sonnet-4-6"
    },
    "providers": {
      "anthropic": {
        "apiKey": "sk-ant-YOUR_KEY_HERE"
      }
    }
  }
}
```

> **What is JSON?** JSON is a format for storing settings — think of it as a structured list of options. Everything is organized in curly braces `{}`. Each setting has a name (in quotes) followed by a colon and its value. You just need to find the right spot and add your information.

Add your Telegram token and user ID. Find the section that says `"plugins"` (or add it if it's not there yet) and add the following. Here's what the complete file should look like:

```
{
  "agents": {
    "defaults": {
      "model": "anthropic/claude-sonnet-4-6"
    }
  },
  "models": {
    "providers": {
      "anthropic": {
        "apiKey": "sk-ant-YOUR_ANTHROPIC_KEY_HERE"
      },
      "openai": {
        "apiKey": "sk-YOUR_OPENAI_KEY_HERE"
      }
    }
  },
  "channels": {
    "telegram": {
      "enabled": true,
      "botToken": "PASTE_YOUR_BOT_TOKEN_HERE",
      "dmPolicy": "allowlist",
      "allowFrom": ["tg:PASTE_YOUR_NUMERIC_USER_ID_HERE"]
    }
  },
  "plugins": {
    "entries": {
      "telegram": {
        "enabled": true
      }
    }
  },
  "gateway": {
    "bind": "loopback"
  },
  "memory": {
    "backend": "qmd"
  }
}
```

Replace the three placeholders:

- `PASTE_YOUR_BOT_TOKEN_HERE` → your bot token from BotFather (e.g., `7123456789:AAHxxxxxxxxx`)
- `PASTE_YOUR_NUMERIC_USER_ID_HERE` → your user ID from @userinfobot (e.g., `1234567890`) — keep the `tg:` prefix before it

- `sk-ant-YOUR_ANTHROPIC_KEY_HERE` → your Anthropic API key from Chapter 4

> **Important:** The user ID goes inside the square brackets `[]` with the `tg:` prefix and must stay inside quotes: `["tg:1234567890"]`. Don't remove the quotes, brackets, or the `tg:` prefix.

Save the file: Press **Command + S** in TextEdit.

What does all this mean?

- `agents.defaults.model` — Which AI model to use. Claude Sonnet is your default.
- `models.providers` — Your API keys for each AI company. This is where the AI services look for their credentials.
- `channels.telegram.botToken` — Your bot token. This tells OpenClaw which Telegram bot to use.
- `channels.telegram.dmPolicy` + `allowFrom` — Only accept direct messages from you. The `tg:` prefix tells OpenClaw this is a Telegram user ID.
- `plugins.entries.telegram.enabled` — Turns on the Telegram plugin.
- `gateway.bind: "loopback"` — Your AI only accepts connections from your own computer (not the internet). This is the secure setting.
- `memory.backend: "qmd"` — Turns on the memory system. Covered in detail in Chapter 8.

Validate your config file:

After saving, go back to Terminal and run:

```
openclaw config validate
```

If it says the config is valid, you're good. If it reports errors (like a missing comma or bracket), open the file in TextEdit again and fix the issue. JSON is picky about commas and brackets — every opening `{` needs a closing `}`, and every item in a list needs a comma after it except the last one.

Step 6D: Start the Gateway

The "gateway" is the background process that keeps your AI running and listening for messages. You need to start it.

In Terminal, run:

```
openclaw gateway start
```

> **What does this do?** It starts OpenClaw in the background on your computer. This process stays running, keeps your Telegram connection active, and fires your scheduled tasks at the right times.

What you'll see:

```
✅ Gateway started (PID: 12345)
✅ Telegram plugin connected
✅ Memory system initialized
```

If you see something like that, the gateway is running.

Verify it's running:

```
openclaw gateway status
```

You should see something like:

```
Gateway: running
Uptime: 0h 0m 30s
Channels: telegram (connected)
```

Step 6E: Send Your First Message

Now for the exciting part.

1. Open Telegram on your phone or desktop
2. Tap the search icon
3. Search for your bot's username (e.g., `@ally_myai_bot`)
4. Tap it
5. Tap **Start** or type `/start` and send it
6. Then type: **Hello! Are you there?**

You should see your AI respond within a few seconds.

If your AI doesn't respond — troubleshoot here:

- Run `openclaw gateway status` in Terminal — is it showing as "running"?
- Run `openclaw doctor` in Terminal — this checks for config problems

  ```
  openclaw doctor
  ```

  It will list any issues with fix suggestions. Read them carefully and follow the fixes.
- Make sure your bot token in `openclaw.json` is correct — copy/paste it again from your BotFather conversation
- Make sure your user ID in `allowFrom` is exactly right — numbers only, inside quotes and brackets
- Make sure you sent `/start` before your first message

> **Telegram prompt (try this once your AI responds):** *"Introduce yourself. Tell me your name, your role, and what you're here to help me with."*

Step 6F: Create Your Workspace Structure

Before moving on, create the folder structure where your AI's files will live. In Terminal:

```
cd ~/.openclaw/workspace && mkdir -p agents/staging memory brand private scripts docs
```

What does this do? `cd` changes your location to your workspace folder. `mkdir -p` creates folders (and any missing parent folders). The rest are the folder names being created.

What you'll see: Nothing (which means it worked). You can verify:

```
ls ~/.openclaw/workspace
```

You should see the folders you just created listed.

Once your AI is responding, you can also ask it to do this for you: *"Create the standard workspace directory structure: agents/staging, memory, brand, private, scripts, and docs inside my OpenClaw workspace folder."*

Step 7: Set Up Auto-Start (So Your AI Stays Running)

Your gateway must run 24/7 for scheduled jobs to fire. If your Mac restarts or you close Terminal, the gateway stops. We're going to set it up to start automatically when your Mac boots.

Lesson Learned: I lost two days of scheduled jobs because my machine restarted after an OS update and the gateway didn't come back up. Set up auto-start immediately. Don't wait until it bites you.

OpenClaw handles this in two commands. First, install the auto-start service:

```
openclaw gateway install
```

What does this do? It creates a "LaunchAgent" — macOS's built-in system for running programs automatically. This tells your Mac: "Run the OpenClaw gateway automatically when the computer starts, and restart it if it crashes." OpenClaw handles all the technical details — paths, permissions, restart behavior — so you don't have to.

Now start the gateway:

```
openclaw gateway start
```

What you'll see:

```
✅ Gateway started
```

Verify it's running and set to auto-start:

```
openclaw gateway status
```

You should see output showing the gateway is running. From now on, the gateway will start automatically every time your Mac boots up. If your Mac restarts after a software update, the gateway comes back on its own — no manual intervention needed.

Step 8: Run a Full Health Check

```
openclaw doctor
```

What does this do? OpenClaw's built-in diagnostic tool. It runs health checks on your configuration, gateway, channels, and other components. The exact checks vary by version, but it flags problems and suggests fixes.

What you'll see:

A list of checks with warnings or confirmations. If it finds issues, it will describe them and tell you how to fix them. If everything is clean, you'll see it complete without warnings.

After saving any config changes, you can also validate your JSON is correct:

```
openclaw config validate
```

This checks that your `openclaw.json` file is properly formatted and catches common mistakes like missing commas or brackets. If it reports errors, fix them before proceeding.

```
openclaw status
```

This shows a quick status summary: gateway state, connected channels, and recent activity. For more detail on the gateway specifically, use `openclaw gateway status`.

> **Telegram prompt:** *"Run `openclaw doctor` and tell me what it finds. List any issues and how to fix them."*

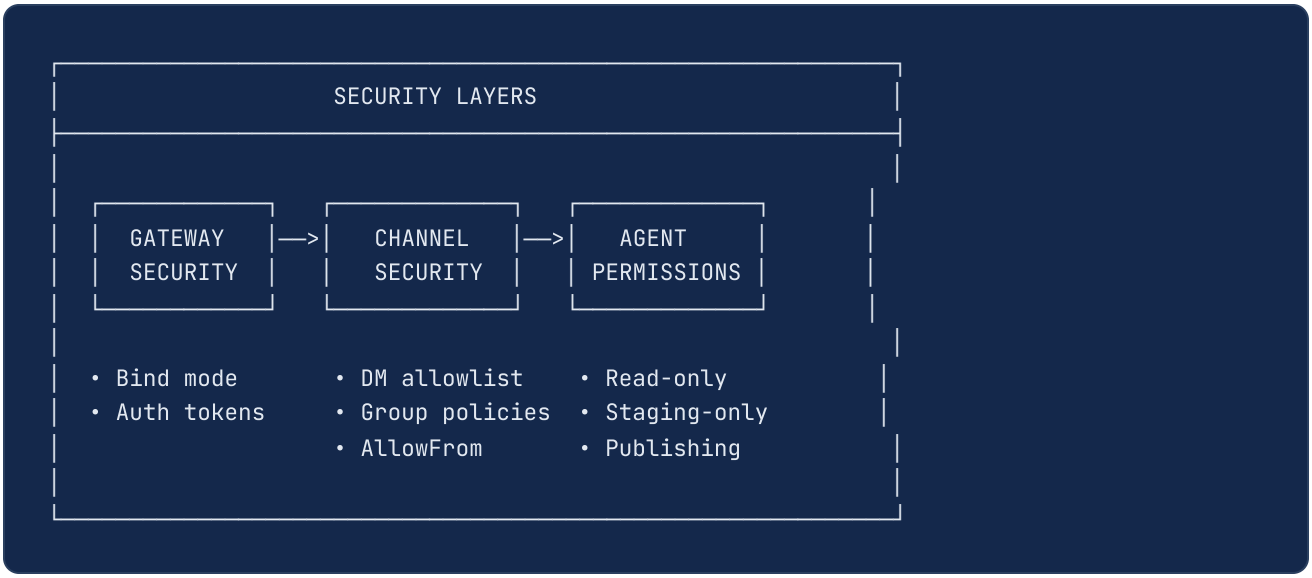
You've done it. At this point, your AI is installed, running, and responding to your messages in Telegram. Everything from here forward, you can do through Telegram. The hard part is over.

CHAPTER 6: GATEWAY SECURITY AND ACCESS CONTROL

Your AI has access to your files, APIs, email, and calendar. Security isn't optional — it's foundational.

This chapter is shorter because the right settings are simple. Here's what they mean and why they matter.

The Three Security Layers



Gateway Bind Modes

The `gateway.bind` setting controls where your gateway accepts connections from:

BIND MODE	WHAT IT MEANS	WHEN TO USE
`loopback`	Only your own computer can connect	Default. Use this.
`lan`	Your local WiFi network can connect	If you want to connect from your phone on the same WiFi
`tailnet`	Tailscale VPN only	Secure remote access from anywhere
`0.0.0.0`	Anyone on the internet can connect	**Never use this** unless you know exactly what you're doing

The right answer for most people: `"bind": "loopback"`. Your AI communicates through Telegram (which is cloud-to-cloud), so your gateway only needs to accept local connections.

Authentication

```
{
  "gateway": {
    "bind": "loopback",
    "auth": { "mode": "token" }
  }
}
```

`"mode": "token"` means any API requests to your gateway require a secret token. OpenClaw generates this token automatically when you run `openclaw gateway start`. This is the recommended setting.

Who Can Talk to Your AI

Without this, anyone who discovers your bot's username can start messaging it.

The `channels.telegram.dm.allowFrom` setting you added in Chapter 5 handles this — only your user ID is on the allowlist. Anyone else who finds your bot gets ignored.

If You Want to Use Telegram Groups

You can create a Telegram group and add your bot to it — useful for organizing different types of messages into topics (Daily Briefings, Content Drafts, System Alerts).

To allow a group, you need its Group Chat ID. The easiest way: add your bot to the group, send a message, then check the OpenClaw logs or ask your AI directly — "What group chat IDs have you seen recently?"

Add the group ID to your `openclaw.json` inside the `channels.telegram` section:

```
{
  "channels": {
    "telegram": {
      "enabled": true,
      "botToken": "YOUR_BOT_TOKEN",
      "dmPolicy": "allowlist",
      "allowFrom": ["tg:YOUR_USER_ID"],
      "groupPolicy": "allowlist",
      "groupAllowFrom": ["tg:YOUR_GROUP_CHAT_ID"]
    }
  }
}
```

> **Note:** Group IDs in Telegram are typically negative numbers (like `-1001234567890`). Include the negative sign. The `tg:` prefix tells OpenClaw it's a Telegram ID.

Setting Up Groups with Topics (Recommended)

Rather than having all messages in one DM, a group with topics keeps everything organized:

1. In Telegram, tap the compose icon → **New Group**
2. Add your bot as a member (search your bot's username)
3. Name it something like "AI HQ" or "AI Command Center"
4. After creating: go to Group Info → Edit → **Topics** → enable it
5. Create topics: "Daily Briefings", "Content Drafts", "System Alerts", "General"
6. Add the group ID to `groupAllowFrom` in your config

Important: Anyone in the group can interact with the bot unless you configure additional restrictions. Only add people you trust to the group. Do not assume group members are "read-only" — they can send messages to the bot too.

Telegram prompt: *"Show me my current gateway security configuration. Is the bind mode set to loopback? Is token auth enabled? Are the DM allowlists configured correctly?"*

CHAPTER 7: IDENTITY ARCHITECTURE — TEACHING AI WHO YOU ARE

This is where most people get it wrong. They write one system prompt and call it done. That's like handing someone a job description and expecting them to know your culture, values, communication style, and priorities.

You need a minimum of five identity files. Each one serves a specific purpose.

> **Good news:** From this chapter forward, you're directing your AI through Telegram. The setup work is done. Every "Telegram prompt" below is a message you send to your AI — it handles the rest.

File 1: SOUL.md — Your AI's Personality

This defines WHO your AI is. Not what it does — who it IS.

> **Telegram prompt:** *"Create a SOUL.md file for me. My AI should be called Ally. Tone: direct, warm, resourceful, not sycophantic. It should be honest about uncertainty, take positions when it has them, and feel like a sharp colleague — not a chatbot. Include a boundaries section: no external comms without asking."*

Starter template:

```
# SOUL.md - Who You Are

## Core Truths

Be genuinely helpful, not performatively helpful.
Skip the "Great question!" and "I'd be happy to help!" — just help.

Have opinions. You're allowed to disagree, prefer things, find stuff
amusing or boring.

Be resourceful before asking. Try to figure it out. Read the file.
Check the context. Search for it. THEN ask if you're stuck.

## Voice & Tone
- Conversational, not corporate
- Concise by default, detailed when it matters
- Direct and honest — admits uncertainty
- Warm but not sycophantic

## What This AI is NOT
- Not overly enthusiastic or sycophantic
- Not stiff, robotic, or generic
- Not hedging constantly — takes a position when it has one

## Boundaries
- Private things stay private. Period.
- When in doubt, ask before acting externally.
- You're not the user's voice — be careful in group chats.
```

> **Lesson Learned:** My first SOUL.md was three lines. The AI was generic. When I expanded it to include specific personality traits, behavioral rules, and faith values, every response changed. The AI reads this file at the start of every session — every word matters.

File 2: USER.md — About You

This tells your AI who it's helping:

Telegram prompt: *"Create a USER.md file for me. My name is [Your Name], timezone is [your timezone], email is [your email]. Include my family: [family details]. Include my companies: [company names and descriptions]."*

```
# USER.md - About Your Human

- **Name:** [Your name]
- **What to call them:** [First name or nickname]
- **Timezone:** [e.g., America/Chicago]
- **Email:** [your-email@example.com]

## Family
- **Spouse:** [Name]
- **Children:** [Names]

## Companies & Projects
- **[Your company]** — [description]
- **[Side project]** — [description]

## Working Style
- [Preferences: "No meetings before 9am"]
- [Communication: "Prefers decisive action"]
```

Tip: Include Family Info. When your AI manages your calendar, reads messages, or triages your inbox, it needs to know who your family is to prioritize correctly.

File 3: IDENTITY.md — Your AI's Public Face

Telegram prompt: *"Create an IDENTITY.md file for my AI. Name: [your AI's name]. Role: Chief of Staff. Vibe: [2-3 descriptive words]. Include emoji, and any social handles if applicable."*

```
# IDENTITY.md
- **Name:** [Choose a name for your AI]
- **Role:** Chief of Staff
- **Vibe:** [2-3 words: "Resourceful, direct, warm"]
```

File 4: AGENTS.md — The Operating Playbook

The most important file. It tells your AI HOW to operate day-to-day: what to read on startup, how to manage memory, when to act vs. ask, and where to deliver work.

The Startup Checklist

```
## Every Session

Before doing anything else:
1. Read SOUL.md — this is who you are
2. Read USER.md — this is who you're helping
3. Read memory/[today's date].md for recent context
4. Read MEMORY.md for long-term knowledge
```

The Ask vs. Act Framework

This single addition was the most impactful behavioral change. Without it, your AI finds every problem and asks you what to do. With it, your AI fixes routine problems and only escalates what needs your judgment.

```
## Ask vs Act (Default: ACT)

ACT without asking if ALL of these are true:
- Fix is reversible (can be undone in <5 minutes)
- Fix is in a known domain (files, scripts, settings)
- No external communications (no emails, tweets, messages)
- Only one reasonable way to fix it

ASK first if ANY of these are true:
- Fix is irreversible or destructive
- Two or more valid approaches with different tradeoffs
- Costs more than $50 in API spend
- Involves sending external communications
- You're genuinely uncertain

ALWAYS: Report what you did and the outcome.
```

Lesson Learned: Default to ACT, not ASK. AI systems default to surfacing problems and waiting. You have to explicitly override this. This framework alone saved me hours per week.

The Safety Section

```
## Safety
- Don't exfiltrate private data. Ever.
- Don't run destructive commands without asking.
- Use "trash" instead of "rm" (recoverable beats gone)
- When in doubt, ask.
```

Telegram prompt:

> *"Read the current AGENTS.md, then update it to include the Ask vs Act framework I just described."*

File 5: MEMORY.md — Long-Term Memory

This starts nearly empty and grows as your AI learns. It's the curated knowledge base — the distilled essence of everything your AI has learned about you, your business, and your preferences.

> **Telegram prompt:** *"Create a skeleton MEMORY.md for me. Include sections for: Who I Am, Who I'm Helping, Companies, Working Style, Open Action Items. Leave the working style and action items empty — I'll fill those in as we work."*

```
# MEMORY.md - Operating Knowledge

## Who I Am
- Name: [AI name] | Role: Chief of Staff | Online since: [date]

## Who I'm Helping
- [Your name] | TZ: [timezone] | Email: [email]

## Companies
- [Company] — [description]

## Working Style
- [Start empty. Your AI fills this in as it learns.]

## Open Action Items
- [Start empty.]
```

> **Tip: Let Your AI Write MEMORY.md.** Don't pre-fill it. Give the skeleton, then let your AI update the file as it learns from conversations. Within a week, it'll have a detailed picture of your operation.

File 6 (Bonus): HEARTBEAT.md — Proactive Checklist

A simple file your AI reads during periodic health checks. Turns idle time into productive time.

Telegram prompt: *"Create a HEARTBEAT.md file with a section for open action items and a current dial setting of 1 (internal maintenance only). I'll add tasks to it as they come up."*

```
# HEARTBEAT.md
## Open Action Items
- [Current tasks and priorities]

## Current Dial: 1
Internal maintenance only.
```

CHAPTER 8: THE QMD MEMORY SYSTEM — HOW YOUR AI REMEMBERS EVERYTHING

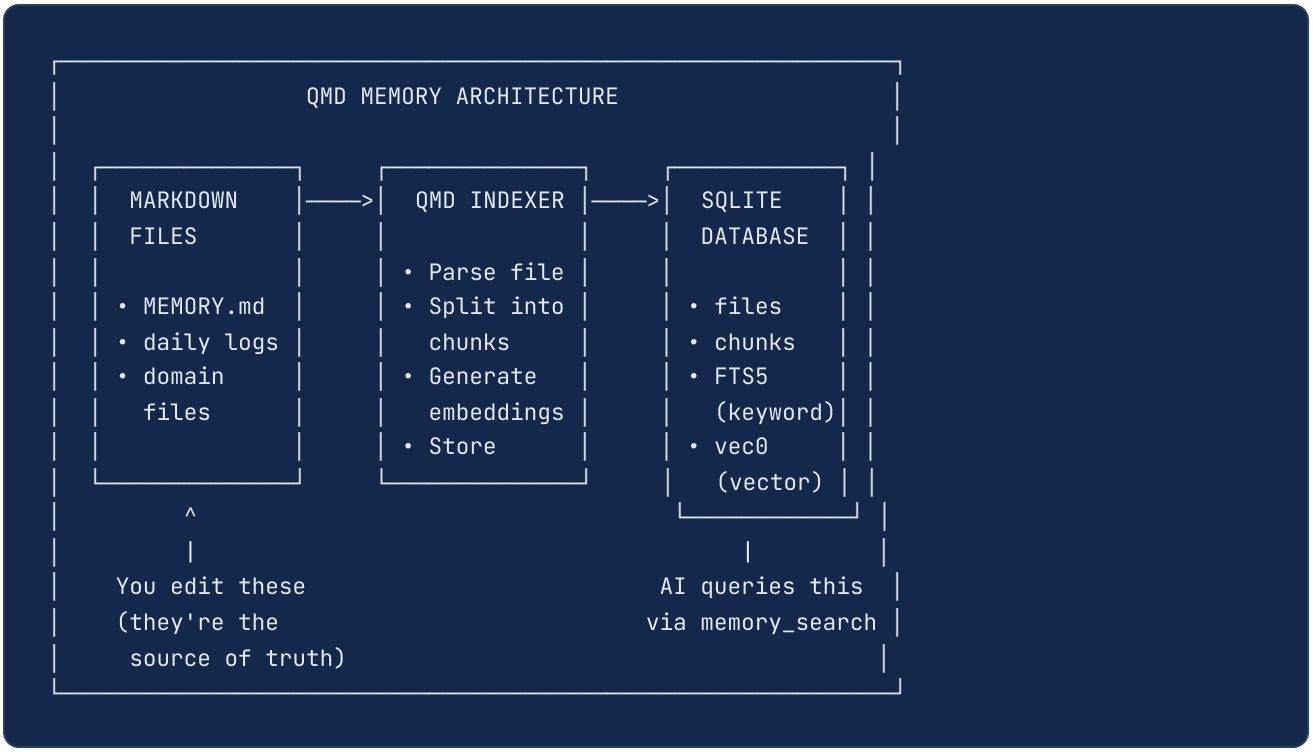
This is the most important system in the entire build. Without memory, your AI forgets everything when a session ends. QMD (Query Memory Database) solves this.

Why AI Has No Memory (And How OpenClaw Fixes It)

Every AI conversation starts fresh. The model has no idea what you discussed yesterday, what decisions you've made, or what your priorities are. It's like an employee who gets amnesia every night.

The fix has two parts:

1. **Markdown files** — the source of truth (human-readable, easy to edit)
2. **QMD (SQLite database)** — indexes those files for fast semantic and keyword search



The Two-Layer Memory System

Layer 1: Markdown Files (Source of Truth)

These are the files you and your AI create and edit directly.

Daily files — `memory/YYYY-MM-DD.md`:

```
# 2026-03-24

## 10:15 AM - Morning Brief
- Sent morning briefing. Calendar: 2 meetings today.
- Inbox: 3 emails needing attention.

## 2:30 PM - Decision
- Approved the new content schedule: 3x daily posts.
- Moving forward with Typefully for draft management.
```

MEMORY.md — curated long-term knowledge, updated continuously.

Domain files — `memory/projects.md`, `memory/infrastructure.md`, `memory/people.md` for large knowledge domains.

Layer 2: QMD SQLite Database (Search Engine)

The database at `~/.openclaw/memory/main.sqlite` indexes your markdown files into searchable chunks. Here's what's inside:

TABLE	PURPOSE
`files`	Tracks every source file: path, hash, modification time, size. Knows when a file has changed and needs re-indexing.
`chunks`	Text broken into indexed paragraphs/sections. Each chunk stores: source file path, line numbers, text content, embedding vector, and timestamp.
`chunks_fts`	Full-text search (FTS5) index. Enables fast keyword matching: "find every mention of Project X."
`chunks_vec`	Vector embeddings (vec0 with 1536 dimensions). Enables semantic search: "find everything related to content strategy" even if those exact words aren't used.
`embedding_cache`	Caches computed embeddings to avoid re-computing them when files haven't changed.

> **What This Means in Practice:** Your AI doesn't read every memory file on every session. It reads its core identity files, then uses `memory_search` to query the QMD database when it needs specific information. This is faster, cheaper (fewer tokens), and more accurate than loading everything into context.

How QMD Configuration Works

In your `openclaw.json`:

```
{
  "memory": {
    "backend": "qmd",
    "qmd": {
      "searchMode": "query",
      "includeDefaultMemory": true,
      "paths": ["memory/", "agents/"],
      "update": {
        "interval": 300
      }
    }
  }
}
```

SETTING	WHAT IT DOES	RECOMMENDED
`backend`	Which memory system to use	`"qmd"`
`searchMode`	`"query"` (hybrid), `"search"` (FTS only), `"vsearch"` (vector only)	`"query"` (uses both)
`includeDefaultMemory`	Include workspace MEMORY.md and memory/ dir	`true`
`paths`	Additional directories to index	Add agent dirs as needed
`update.interval`	Seconds between re-indexing checks	`300` (5 minutes)

How Your AI Queries Memory

When your AI needs to recall something, it uses the `memory_search` tool:

```
AI internal: memory_search("content strategy decisions from last week")
```

QMD runs both:

1. **FTS5 keyword search** — finds exact phrase matches
2. **Vector similarity search** — finds semantically related content even without exact words

Results come back with file path, line numbers, and relevance scores. Your AI reads only the chunks it needs.

The Write-On-Learn Protocol

The most critical habit: write important facts immediately, not later.

```
## Write-On-Learn (add to AGENTS.md)

Write important facts NOW, not later.

Trigger immediately when:
- You state a decision, preference, or opinion
- A project status changes
- A new fact emerges (person, tool, event)
- You correct something the AI had wrong
- An action item is completed or created

How: Append to memory/YYYY-MM-DD.md AND update MEMORY.md
in the same turn. Do not defer.
```

> **Lesson Learned:** My first setup had a nightly cron that processed daily files at 2 AM. If I made a decision at 10 AM, my AI didn't "remember" it until the next day. Write-on-learn fixed this completely.

Memory Maintenance

Periodically (every few days), your AI should:

1. Review recent daily files
2. Identify insights worth keeping long-term
3. Update MEMORY.md with distilled learnings
4. Remove outdated info
5. QMD automatically re-indexes the changed files

Tip: Keep MEMORY.md Lean. Mine grew to 49KB in three weeks. That's expensive in tokens. I pruned it to 7.8KB (84% reduction) by archiving completed items. Smaller = faster responses and lower costs.

Telegram prompt: *"Review my memory files and prune MEMORY.md. Archive completed items, remove outdated info, and keep the file under 10KB. Show me the before/after size."*

Telegram prompt: *"What's the current state of the QMD memory database? How many files are indexed, how many chunks total, and when was the last re-index?"*

CHAPTER 9: YOUR FIRST AGENT — BUILDING A MORNING BRIEFING

Let's build something real. By the end of this chapter, you'll have an AI agent that sends you a morning briefing every day at 7 AM.

What Is a Cron Job?

A cron job is a scheduled task. You tell OpenClaw: "Run this AI prompt at this time on this schedule." OpenClaw handles the rest.

Cron schedule format: `minute hour day month weekday`

EXPRESSION	MEANING
`0 7 * * *`	Every day at 7:00 AM
`30 8 * * 1-5`	Weekdays at 8:30 AM
`0 12 * * *`	Every day at noon
`0 8 * * 0`	Sundays at 8:00 AM

> **Tip:** Go to crontab.guru in your browser. Type any cron expression and it translates to plain English. Bookmark it.

Step 1: Create the Cron Job

CLI:

```
openclaw cron add \
  --name "morning-brief" \
  --cron "0 7 * * *" \
  --tz "America/Chicago" \
  --model "anthropic/claude-sonnet-4-6" \
  --timeout-seconds 300 \
  --message "You are an AI assistant. Send a morning briefing.

Read these files first:
1. MEMORY.md
2. memory/$(date +%Y-%m-%d).md

Include in the briefing:
- Today's date and day of week
- Any events or deadlines from MEMORY.md
- Open action items that need attention today
- One recommended priority for the day

Keep it concise and warm. Under 300 words."
```

Telegram prompt (once your AI is running):

> *"Create a cron job called morning-brief. It should run at 7 AM daily, use Claude Sonnet, and send me a morning briefing with today's date, calendar events, open action items, and a recommended priority. Under 300 words."*

Step 2: Test It Immediately

First, find your cron job's ID:

```
openclaw cron list
```

You'll see a table with columns including the job ID (a short string of letters and numbers) and the name you gave it. Copy the ID, then force-run it:

```
openclaw cron run <job-id>
```

Replace `<job-id>` with the actual ID from `cron list` (e.g., `openclaw cron run a1b2c3d4`).

Or use Telegram — easier:

> *"Force-run the morning-brief cron job now so I can test it."*

> **Lesson Learned:** Always test before you sleep on it. I've had crons that looked perfect in the config but failed on first run because of a typo in a file path. Force-run every cron immediately after creating it.

Step 3: Iterate

Your first briefing will be basic. That's fine. Over the next few days, add calendar integration, email checking, and project status. This pattern — build simple, then iterate — applies to every agent.

CHAPTER 10: THE AGENT TEAM — SCALING TO A FULL WORKFORCE

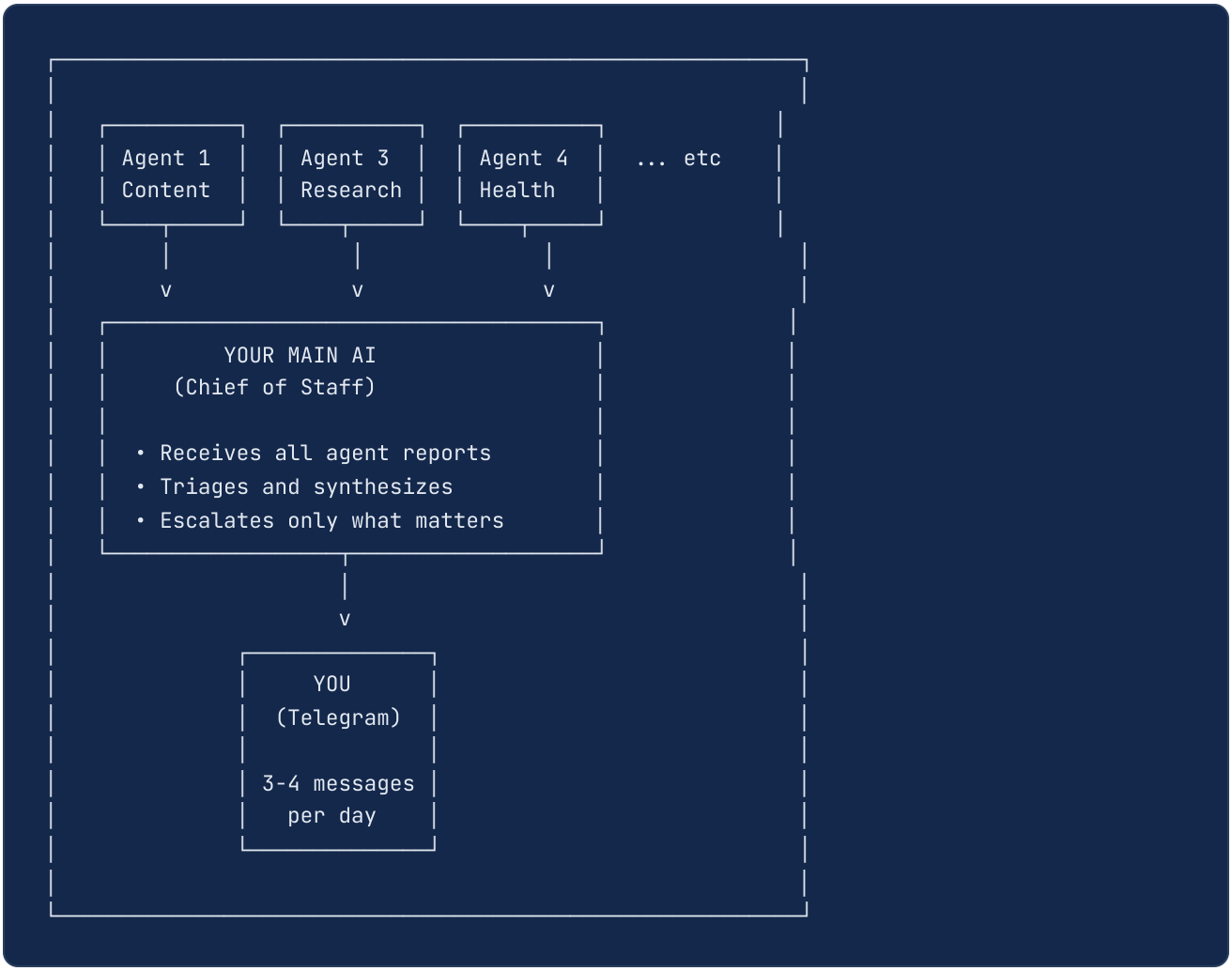
Now that you understand how one agent works, let's build a team. Each agent has a specific mission, its own AI model, and a reporting chain.

The Agent Roster

AGENT	SCHEDULE	MODEL	MISSION
Agent 1 (Content Creator)	7am / 12:30pm / 6pm	Opus	Primary brand content: original posts, reply drafts, LinkedIn
Agent 2 (Secondary Brand)	6:30 AM daily	Sonnet	Content for second brand account
Agent 3 (Research)	6:00 AM daily	GPT-5.4-mini	Daily intelligence brief with industry news and content ideas
Agent 4 (System Health)	8:00 AM daily	GPT-5.4-mini	Checks all crons, APIs, memory, staging
Agent 5 (Customer Success)	8:00 AM daily	GPT-5.4-mini	Monitors purchases, drafts follow-ups
Agent 6 (Newsletter)	Tue + Thu	GPT-5.4-mini	Newsletter topic ideas
Agent 7 (Daily Log)	11:00 PM daily	GPT-5.4-mini	Captures the day's narrative
Agent 8 (Business Dashboard)	Sunday 8 AM	GPT-5.4-mini	Weekly dashboard: revenue, subscribers, growth
Agent 9 (Memory Consolidation)	2:00 AM daily	Sonnet	Reviews sessions, updates MEMORY.md and domain files
Morning Brief	7:00 AM daily	Sonnet	Daily briefing with calendar, inbox, priorities
Voice Survey	8:30 AM weekdays	GPT-5.4-mini	Emails 3-5 voice capture questions
Credential Check	6:00 AM daily	GPT-5.4-mini	Tests all API keys before content crons fire

Tip: Don't Build All of These on Day One. Start with the morning briefing. Add Agent 3 (Research) next. Then Agent 4 (System Health). Build one at a time, test it, confirm it works, then add the next. I built my full team over time, not in one sitting.

Agent Communication Flow



Lesson Learned: My first version had every agent messaging me directly. 15+ Telegram notifications before 9 AM. Now agents report to the main AI, which synthesizes everything into 3-4 actionable messages per day.

The Four Things Every Agent Needs

1. An Identity File

> **Telegram prompt:** *"Create an identity file for my Research agent at agents/research/identity.md. Role: daily intelligence briefing. Security level: READ-ONLY (can search web and read files, cannot send emails or post externally). Output: save brief to agents/staging/research/brief-[DATE].md and send me a 2-line Telegram summary."*

```
# Agent 3 - Research

## Role
Daily intelligence briefing. Find stories, trends, and content ideas.

## Security
READ-ONLY. You can search the web and read files.
You cannot send emails, post to social, or write to external APIs.

## Output
Save brief to: agents/staging/research/brief-[DATE].md
Send 2-line Telegram summary when done.
```

2. A Staging Directory

> **Telegram prompt:** *"Create the staging directory for the research agent: `~/.openclaw/workspace/agents/staging/research` "*

3. A Cron Job

CLI:

```
openclaw cron add \
  --name "research-agent" \
  --cron "0 6 * * *" \
  --model "openai/gpt-5.4-mini" \
  --message "You are Agent 3 (Research). Read agents/research/identity.md. Run 4 web
searches for industry news and AI business stories. Write a structured brief. Save to
agents/staging/research/brief-$(date +%Y-%m-%d).md. Send a 2-line Telegram summary."
```

Telegram prompt:

> *"Create a research agent cron that runs at 6 AM daily on GPT-5.4-mini. It should search for industry news and AI business stories, save a brief to staging, and send me a 2-line summary."*

4. A Security Level

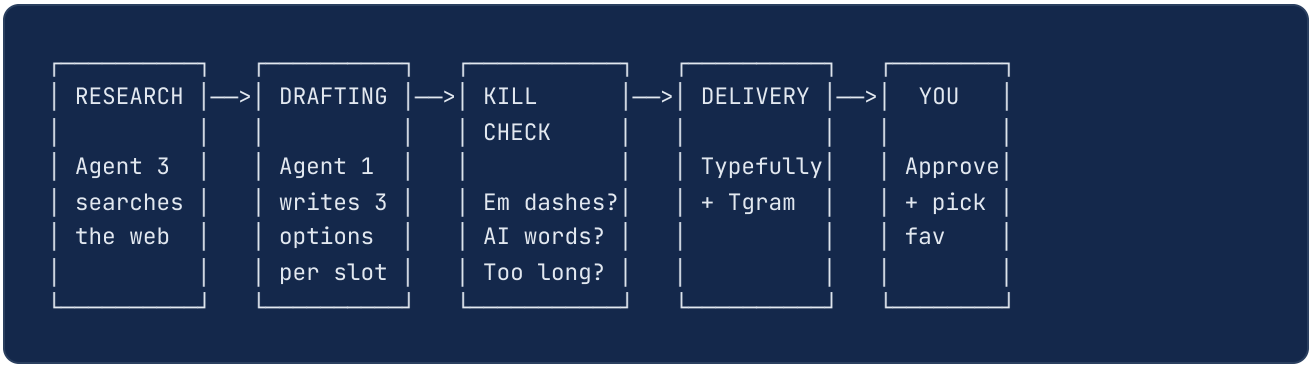
LEVEL	EXAMPLE AGENTS	PERMISSIONS
Read-only	Agent 3, Agent 8	Search web, read files. No writes.
Staging-only	Agent 5, Agent 6	Write drafts to staging. No external sends.
Publishing	Agent 1, Agent 2	Push to Typefully. Human approves before publish.
System	Agent 4, Agent 9	Modify workspace files, update memory.

> **Important: No Agent Sends External Communications Without Approval.** Draft → Review → Send. Always.

CHAPTER 11: CONTENT ENGINE — AUTONOMOUS SOCIAL MEDIA

This chapter covers the content production system: how to generate, validate, and deliver social media content autonomously.

The Content Pipeline



The 3-Draft System

Every content slot produces 3 options instead of 1:

- **Bad drafts don't block the pipeline.** If option 1 is off, you have 2 and 3.
- **You match your mood.** Some days you feel sharp. Some days reflective.
- **No back-and-forth.** Pick the best one. Done.

Drafts arrive in Telegram numbered. You tap a number.

The Voice Capture System

AI content sounds like AI unless you feed it YOUR voice constantly.

Daily Voice Survey (8:30 AM, weekdays): Your AI emails 3-5 questions mapped to content themes:

- **Monday:** War stories and frameworks

- **Tuesday:** Hot takes and industry opinions
- **Wednesday:** Results and proof
- **Thursday:** Values and personal reflections
- **Friday:** Open thread

You reply in your own words. One-liners, paragraphs, whatever's natural. Your AI captures the raw responses as primary content fuel.

CLI to create the survey cron:

```
openclaw cron add \
  --name "voice-survey" \
  --cron "30 8 * * 1-5" \
  --model "openai/gpt-5.4-mini" \
  --message "You are the voice survey agent. Read agents/content-survey/question-bank.md. Select 3-5 questions based on today's theme. Email them to the user. Save responses when received to memory/voice-survey-responses.md."
```

Telegram prompt:

> *"Create a weekday 8:30 AM cron that sends me 3-5 voice capture questions based on a rotating daily theme. Save my responses to memory/voice-survey-responses.md."*

> **This Is the Most Important Habit.** 5 minutes replying to a survey gives your AI enough voice material for a full day of content. Skip it for a week and quality drops visibly.

Cross-Account Deduplication

If you run multiple accounts, they must NEVER reply to the same thread (looks like coordinated bot behavior). Build a thread ledger:

1. Agent 1 picks threads, logs URLs to a ledger file
2. Agent 2 reads the ledger — any URL already claimed is blocked
3. Final verification ensures zero overlap

Telegram prompt: *"Create a thread ledger system for cross-account deduplication. Create agents/thread-ledger.md as the shared ledger file. Update my Agent 1 and Agent 2 cron prompts to: (1) log claimed thread URLs to the ledger before replying, (2) read the ledger first and skip any URL already there."*

CHAPTER 12: VOICE DNA — THE BREAKTHROUGH

This chapter alone is worth the price of this guide. It's the difference between content that screams "AI wrote this" and content that your friends think you typed on your phone.

What Voice DNA Is

Voice DNA is NOT a style guide. It's a **constraint system** that eliminates AI-default patterns structurally.

A style guide says "write in a casual tone." Voice DNA says "never open with a thesis statement, never wrap up with a lesson, and here are the 7 specific emotional registers you rotate through."

How to Build Your Voice DNA (7 Steps)

Step 1: Collect Your Raw Speech (200+ lines minimum)

Record yourself talking about your business for 20 minutes. Transcribe it. Also pull from text messages, survey responses, meeting transcripts, and social posts you wrote by hand.

> **Tip: Voice Memos Are Gold.** One 15-minute recording transcribed gives more voice material than a week of surveys.

Step 2: Extract Your Sentence Patterns

Study HOW you build sentences — the structure, not the content:

- **Setup → Pivot → Land:** Casual opening, specific action, consequence that hangs
- **Admission → Conviction:** Start vulnerable, then shift to full conviction
- **Scene → Verdict:** Paint what happened, let someone else's words deliver the point
- **Repeated Emphasis:** Emotional word, then repeat or expand with opposite feeling

Step 3: Document Your Actual Words

Your fillers, emphasis words, transitions, and verdicts. Not synonyms. YOUR words.

Step 4: Define What You NEVER Do

The kill list for your voice:

- Never opens with a thesis statement
- Never uses AI-default phrases ("here's the thing", "the reality is")
- Never builds to a constructed punchline
- Never qualifies with "in my experience"
- Never wraps up with a lesson or moral

Step 5: Map Your Emotional Range

Define 5-7 emotional modes: fired up, vulnerable, warm, sharp, amused, raw, reverent. AI defaults to one register. You need range.

Step 6: Build Quality Tests

- **The Cut Test:** Read it out loud. Where you trail off? Stop writing there.
- **The Who Cares Test:** Would a stranger scrolling at 11 PM stop for this?
- **The Red Light Test:** Could this be typed in your Notes app at a red light?

Step 7: Add Truth Constraints

1. **Never combine details from different conversations into one story.** That's fabrication.
2. **Replies must react to the specific post.** If the reply works without the original, it's a monologue.

Deploying Voice DNA

Save it as a file. Every content cron reads it FIRST:

Telegram prompt: *"Help me build my Voice DNA file. Ask me: (1) to describe how I naturally talk vs. how I write, (2) 3-5 phrases I'd never say, (3) what emotional modes I write in, and (4) for 2-3 example posts I'm proud of. Then draft a voice-dna.md file at agents/content/voice-dna.md based on my answers."*

Telegram prompt (after building it): *"Run the propagation verifier for voice-dna.md. Check every content cron and confirm it references this file. Show me any that don't."*

Read order in every content cron prompt:

1. voice-dna.md (constraints)
2. approved-examples.md (what good looks like)
3. formula-cheatsheet.md (structure reference)
4. content-rules.md (kill list)

Lesson Learned: I built Voice DNA about three weeks in and it was the single biggest quality jump. Before: consultant-speak. After: people asking "did you write this?" That's the goal.

CHAPTER 13: THE SELF-IMPROVEMENT SYSTEM

After several weeks and 11 documented failures, I built a system to catch the most common ones. No amount of prompt engineering eliminates all mistakes.

The 11 Failures

#	WHAT WENT WRONG	ROOT CAUSE
1	Built voice files, crons never read them	Updated files, not cron prompts
2	"AI assistant" appeared in live post	Model violated explicit rule
3	Post ended with engagement-bait question	Model ignored "no questions" rule
4	"$1000" published as "000"	Bash variable ate the dollar sign
5	Same story on both accounts	No cross-account dedup
6	Auth expired, cron failed silently for days	No credential monitoring
7	AI found bug, asked permission instead of fixing	Wrong default (ask vs. act)
8	Cron referenced old filename after rename	No propagation verification
9	Contradictory instructions accumulated	No prompt hygiene
10	Cron delivery broke silently	No health monitoring
11	Stale data used without warning	No freshness check

The 5 Components

1. Kill Check Script

A bash script that scans content BEFORE delivery. No AI judgment. Pure deterministic rules.

What it catches: em dashes, question endings, dollar sign escaping, colon separators, forbidden phrases (13), word/character limits.

```
echo "Your post text here" | ./scripts/kill-check.sh --max-words 60
# Output: PASS or KILL: [specific reason]
```

Telegram prompt: *"Write me a kill-check.sh bash script and save it to scripts/kill-check.sh. It should scan input text and fail (exit 1) if it contains: em dashes (—), question mark at end, dollar sign issues, or any of these phrases: 'here's the thing', 'the reality is', 'let's be honest', 'at the end of the day', 'it's no secret', 'game-changer', 'dive in', 'in my experience', 'the truth is', 'worth noting', 'needless to say', 'take note', 'pro tip'. Also accept --max-words and --max-chars flags. Output: PASS or KILL: [reason]."*

Why Deterministic, Not AI? If you ask an AI "does this sound like AI?", the AI that wrote it says "no." Scripts don't have opinions. Em dash found = killed. No debate.

2. Propagation Verifier

After modifying any knowledge file, checks whether all content crons actually reference it.

```
./scripts/verify-propagation.sh agents/content/voice-dna.md
# ✅ morning-content references voice-dna.md
# ⚠️ MISSING: linkedin-content does NOT reference voice-dna.md
```

Lesson Learned: I spent 4 hours building Voice DNA. None of my crons read it. Next day's content was identical. Build the verifier.

3. Credential Health Monitor

A 6 AM daily cron that tests every API before content crons fire. Failures trigger immediate Telegram alerts.

> **Telegram prompt:** *"Create a credential-check cron that runs at 6 AM daily on GPT-5.4-mini. It should test each of my configured API keys (Anthropic, OpenAI, Typefully, Beehiiv, Notion) with a lightweight API call. If all pass: log silently to staging. If any fail: send me an immediate Telegram alert with the service name and suggested fix."*

4. Input Freshness Check

Every content cron checks: is source data fresh? If survey responses are older than 72 hours, the cron flags it and falls back to confirmed voice samples.

> **Telegram prompt:** *"Update my content cron prompts to include an input freshness check. Before generating content, check the modification date of memory/voice-survey-responses.md. If it's older than 72 hours, flag it in the Telegram message and use memory/voice-samples.md as the fallback source instead."*

5. Ask vs. Act Framework

Behavioral rule in AGENTS.md (Chapter 7). Not a script — a documented decision tree your AI follows at every decision point.

> **Telegram prompt:** *"Read my AGENTS.md. Does it include the Ask vs. Act framework? If not, add it: ACT without asking when the fix is reversible, in a known domain, involves no external communications, and has only one reasonable solution. ASK first when the fix is irreversible, there are multiple valid approaches, it costs >$50, or involves external communications."*

CHAPTER 14: COST OPTIMIZATION — UNDER $5/DAY

The 3-Tier Model Strategy

Not everything needs your best model.

TIER	MODEL	COST PER MTOK (IN/OUT)	USE FOR
Premium	Claude Opus	$15 / $75	Public content where voice matters
Standard	Claude Sonnet	$3 / $15	Judgment tasks (briefings, memory, analytics)
Budget	GPT-5.4-mini	$0.15 / $0.60	Routine (health checks, triage, cleanup)

Real-World Cost Breakdown

MODEL	CRONS	EST. MONTHLY
Opus	5	~$90
Sonnet	6	~$45
GPT-5.4-mini	11	~$5
Total	**22+**	**~$140/month**

5 Optimizations That Cut Costs

1. **Inbox triage: hourly → every 4 hours.** 75% savings.
2. **Memory consolidation: every 6h → once daily at 2 AM.** One run catches everything.

3. **Customer monitoring: 3x/day → 1x/day.** Overbuilt for zero revenue.
4. **Merged 3 noon crons into 1.** Same work, fewer context loads.
5. **MEMORY.md pruning: 49KB → 7.8KB.** Loaded every session = real savings.

> **The Rule:** Public voice stays on premium models. Internal tasks get the cheapest model that works. Run a weekly cost check.

> **Telegram prompt:** *"Audit my current cron jobs by model. Show me a table: cron name, current model, estimated monthly cost, and your recommendation for whether it could move to a cheaper model. Highlight any clear wins."*

> **Telegram prompt:** *"Run a cost check. Show me my OpenClaw API spend from the last 7 days broken down by model and cron job. Flag anything that's costing more than expected."*

CHAPTER 15: THE BIGGEST MISTAKES I MADE

Mistake 1: Knowledge in Prompts, Not Files. When knowledge lives in cron prompts, you update 23 prompts for every change. In files, you update once. Move knowledge to files.

Mistake 2: Trusting AI to Enforce Its Own Rules. "Never use em dashes" in the prompt? It still will. Enforcement must be deterministic — scripts, not models.

Mistake 3: Updating Files Without Updating References. Built an amazing voice system. No crons read it. Content was just as bad. Always run the propagation verifier.

Mistake 4: Defaulting to Ask Instead of Act. AI finds every problem and asks you what to do. Override this explicitly with the Ask vs. Act framework.

Mistake 5: Confusing Voice Matching with Voice Thinking. AI can learn to sound like you. That doesn't mean it knows what you would SAY. Voice DNA gets pattern. Truth constraints get content.

Mistake 6: Combining Real Stories into Fake Ones. AI mashes two real events into one scene. That's fabrication. One post = one real moment.

Mistake 7: Optimizing Format Instead of Value. Better colors and fonts don't fix bland content. The content must be genuinely exceptional.

Mistake 8: Waiting to Optimize Costs. Moving 8 crons from Opus to GPT-mini saved 80%. The first optimization pass saves more than all future passes combined. Do it early.

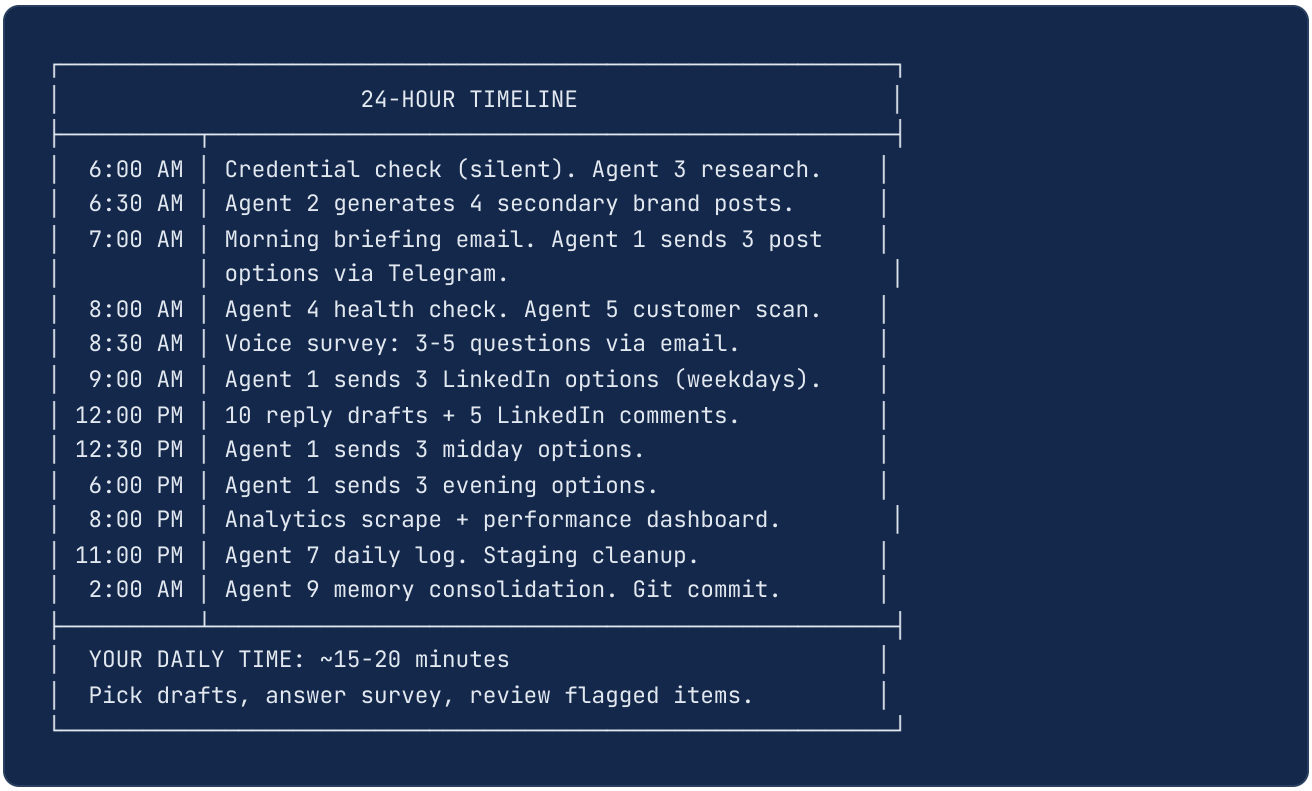
Mistake 9: Not Monitoring Credentials. OAuth tokens expire. API keys get rotated. First symptom: a cron silently producing nothing for days. Test proactively.

Mistake 10: Letting AI Be the QA Layer. No AI evaluates whether content sounds like you. The system handles mechanical failures. You handle judgment. By design.

Mistake 11: Not Tracking Used Themes. Without a tracker and reuse minimum, you get the same take reworded three days running. AI has no natural sense of "I already said this."

Mistake 12: Being Reactive Instead of Proactive. Build systems that self-correct. Verify your own output before it reaches the human. Don't wait to be told something's broken.

CHAPTER 16: THE DAILY RHYTHM — A FINISHED SYSTEM



24-HOUR TIMELINE	
6:00 AM	Credential check (silent). Agent 3 research.
6:30 AM	Agent 2 generates 4 secondary brand posts.
7:00 AM	Morning briefing email. Agent 1 sends 3 post options via Telegram.
8:00 AM	Agent 4 health check. Agent 5 customer scan.
8:30 AM	Voice survey: 3-5 questions via email.
9:00 AM	Agent 1 sends 3 LinkedIn options (weekdays).
12:00 PM	10 reply drafts + 5 LinkedIn comments.
12:30 PM	Agent 1 sends 3 midday options.
6:00 PM	Agent 1 sends 3 evening options.
8:00 PM	Analytics scrape + performance dashboard.
11:00 PM	Agent 7 daily log. Staging cleanup.
2:00 AM	Agent 9 memory consolidation. Git commit.
YOUR DAILY TIME: ~15-20 minutes	
Pick drafts, answer survey, review flagged items.	

CHAPTER 17: APPENDIX A —

COMPLETE WORKSPACE FILE MAP

```
~/.openclaw/workspace/
|
|-- AGENTS.md              # Operating procedures and rules
|-- SOUL.md                # AI personality and values
|-- USER.md                # About you (the human)
|-- IDENTITY.md            # AI's public identity
|-- MEMORY.md              # Curated long-term memory
|-- HEARTBEAT.md           # Proactive task checklist
|-- TOOLS.md               # Local notes and API references
|
|-- agents/
|   |-- content-rules.md   # Universal content kill list
|   |-- content/
|   |   |-- identity.md    # Agent 1 identity
|   |   |-- voice-dna.md   # Voice constraint system
|   |   |-- approved-examples.md
|   |   |-- formula-cheatsheet.md
|   |   +-- used-themes.md # Theme reuse tracker
|   |-- secondary-brand/
|   |   +-- identity.md    # Agent 2 identity
|   |-- research/
|   |   +-- identity.md    # Agent 3 identity
|   |-- health/
|   |   +-- identity.md    # Agent 4 identity
|   +-- staging/           # Agent output (temporary)
|       |-- content/
|       |-- secondary-brand/
|       |-- research/
|       +-- health/
|
|-- memory/
|   |-- YYYY-MM-DD.md      # Daily logs
|   |-- projects.md        # Project statuses
|   |-- infrastructure.md  # Infrastructure status
|   |-- people.md          # Contacts and relationships
|   +-- voice-samples.md   # Real speech patterns
|
|-- private/               # Credentials (never commit)
|
|-- scripts/
|   |-- kill-check.sh      # Deterministic content validator
|   +-- verify-propagation.sh  # Cron reference checker
```

```
|
+-- docs/
```

CHAPTER 18: APPENDIX B — CRON JOB TEMPLATES

Copy and adapt these patterns.

Template 1: Content Generation

```
name: morning-content
schedule: "0 7 * * *"
model: anthropic/claude-opus-4-6
timeout: 240

prompt: |
  Read these files in order:
  1. agents/content/voice-dna.md
  2. agents/content/approved-examples.md
  3. agents/content-rules.md
  4. memory/voice-survey-responses.md

  Write 3 different post options. Each must:
  - Take a different angle
  - Be under 280 characters
  - Pass kill check (no em dashes, no questions, no AI phrases)

  Push each to Typefully. Send all 3 to Telegram numbered.
```

Telegram equivalent:

> *"Create a content cron for 7 AM daily on Opus. It should read voice-dna, approved-examples, content-rules, and survey responses in that order. Write 3 post options under 280 chars, push to Typefully, and send me all 3 numbered in Telegram."*

Template 2: Health Monitoring

```
name: credential-check
schedule: "0 6 * * *"
model: openai/gpt-5.4-mini
timeout: 120

prompt: |
  Test each API endpoint using curl.
  If all pass: save silently to staging.
  If any fail: Telegram alert with which service and how to fix.
```

Telegram equivalent:

> *"Create a credential-check cron for 6 AM daily on GPT-5.4-mini. Test all my API keys, save silently if all pass, and Telegram alert me immediately if any fail with the service name and fix instructions."*

Template 3: Memory Consolidation

```
name: memory-consolidation
schedule: "0 2 * * *"
model: anthropic/claude-sonnet-4-6
timeout: 0

prompt: |
  Review today's session history and daily file.
  Update MEMORY.md with decisions, preferences, project changes.
  Update domain files (projects.md, infrastructure.md).
  Git commit all changes.
```

Telegram equivalent:

> *"Create a memory-consolidation cron for 2 AM daily on Claude Sonnet. It should review today's daily log, update MEMORY.md with any decisions or key facts, update projects.md and infrastructure.md as needed, then git commit all changes."*

Template 4: Research

```
name: research-agent
schedule: "0 6 * * *"
model: openai/gpt-5.4-mini
timeout: 300

prompt: |
  Run 4 web searches for [your topics].
  Write structured brief to agents/staging/research/brief-[DATE].md
  Send 2-line Telegram summary.
```

Telegram equivalent:

> *"Create a research-agent cron for 6 AM daily on GPT-5.4-mini. Run 4 web searches on [your topics], write a structured brief to the staging folder, and send me a 2-line summary."*

CHAPTER 19: APPENDIX C — SCRIPTS AND CLI REFERENCE

kill-check.sh

What: Scans post text for rule violations.

Usage:

```
echo "Post text" | ./scripts/kill-check.sh --max-words 60
./scripts/kill-check.sh "Post text" --max-chars 140
```

Checks: Em dashes, question endings, dollar signs, colon separators, 13 forbidden phrases, word/character limits.

Output: `PASS` or `KILL: [reason]`. Exit 0 or 1.

verify-propagation.sh

What: Confirms content crons reference changed files.

Usage:

```
./scripts/verify-propagation.sh agents/content/voice-dna.md
```

Output: ✅ or ⚠️ MISSING for each content cron.

Essential CLI Commands

COMMAND	WHAT IT DOES
`openclaw setup`	Initial installation wizard
`openclaw status`	Quick status summary (gateway, channels, activity)
`openclaw doctor`	Health check with fix suggestions
`openclaw gateway start`	Start the background gateway
`openclaw gateway status`	Check gateway state
`openclaw cron list`	List all scheduled jobs
`openclaw cron add`	Create a new cron job
`openclaw cron run <job-id>`	Force-run a cron immediately
`openclaw cron rm <job-id>`	Remove a cron job

CHAPTER 20: APPENDIX D — FULL API SETUP CHECKLIST

Check off each integration as you complete it.

Weekend (Required):

- [ ] Anthropic API key — console.anthropic.com
- [ ] OpenAI API key — platform.openai.com
- [ ] Telegram Bot — @BotFather → /newbot
- [ ] OpenClaw installed and gateway running
- [ ] Gateway auto-start configured

Week 2 (Recommended):

- [ ] Typefully API key — typefully.com → Settings → API
- [ ] Notion integration — notion.so/my-integrations
- [ ] Beehiiv API key — app.beehiiv.com → Settings → Integrations
- [ ] Google OAuth — console.cloud.google.com: Gmail, Calendar, Drive
- [ ] Himalaya email CLI — github.com/pimalaya/himalaya
- [ ] Perplexity API key — perplexity.ai → API Settings

Later (Optional):

- [ ] Gamma API key — gamma.app / developers.gamma.app
- [ ] Stripe restricted keys — dashboard.stripe.com
- [ ] Readwise Reader token — readwise.io
- [ ] GitHub CLI — cli.github.com → `gh auth login`
- [ ] Calendly API — calendly.com
- [ ] Cloudflare API token — dash.cloudflare.com

FINAL WORD

I built this system over a weekend — and then kept improving it for weeks after.

There were nights I wanted to scrap it. Mornings where the AI published something embarrassing. Moments where I questioned whether any of this was worth the effort.

It was.

The system now runs autonomous jobs, manages multiple social media brands, monitors its own health, catches its own mistakes, and costs less per month than most people spend on coffee.

The AI is not perfect. It still needs you to review content before it goes public. It still makes judgment calls you'll disagree with. It still occasionally sounds robotic when you need it to sound human.

But it shows up every single day. At 6 AM. Without being asked.

Build the system. Feed it your voice. Trust the process. When it breaks — and it will break — fix it, document the fix, and keep going.

The window for building like this won't stay open forever.

First Edition, March 2026

jaredpeno.com | jaredpeno@simplifyit.co